

101 - Custom authorization requirement and handler example

Към момента, вписаният потребител може сам да си редактира профила и правата. Не искаме да допускаме това. Искаме Админ да може да редактира правомощията на останалите потребители, но не и своите. Това да може да направи само друг Админ. За целта трябва да знаем ID на потребителя, който се е вписал и на този, който се редактира. Ако съвпадат, трябва да се отказва достъп. Към момента, ID на потребителя, който се редактира, се подава в URL адреса като query string parameter.

Custom Requirement & Handler Example

An **Admin** user can manage **other Admin** user roles and claims but not their own claims and roles

Func delegate and **RequireAssertion()**

➤ Part 99 - ASP.NET Core Tutorial

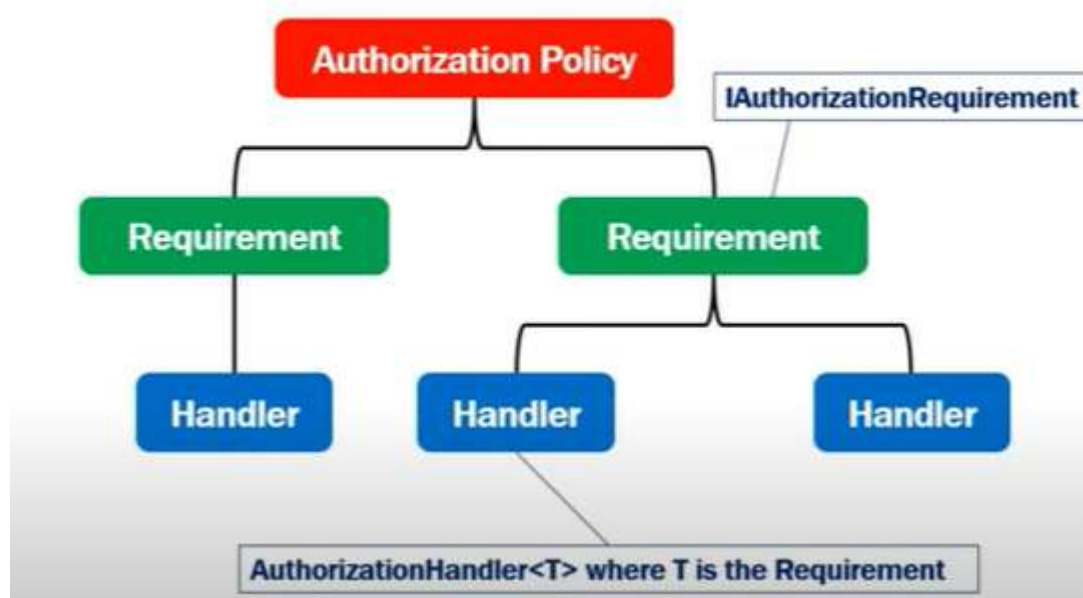
Func cannot be used when

- We need to access **route data** or **query string parameter**
- We need access to other services via **dependency injection**

Трябва да създадем клас и имплементираме интерфейс `IAuthorizationRequirement`.

За целта в основната директория на проекта създаваме отделна папка Security -> Add -> class -> `ManageAdminRolesAndClaimsRequirement.cs`

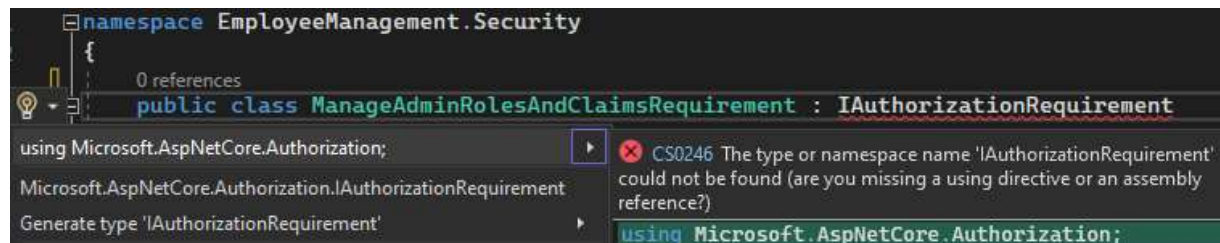
Custom authorization requirement



To create a custom authorization requirement, create a class that implements the [IAuthorizationRequirement](#) interface. This is an empty marker interface, which means there is nothing in this interface that our custom requirement class must implement.

```
public class ManageAdminRolesAndClaimsRequirement : IAuthorizationRequirement
{
}
```

[IAuthorizationRequirement](#) interface is in [Microsoft.AspNetCore.Authorization](#) namespace.



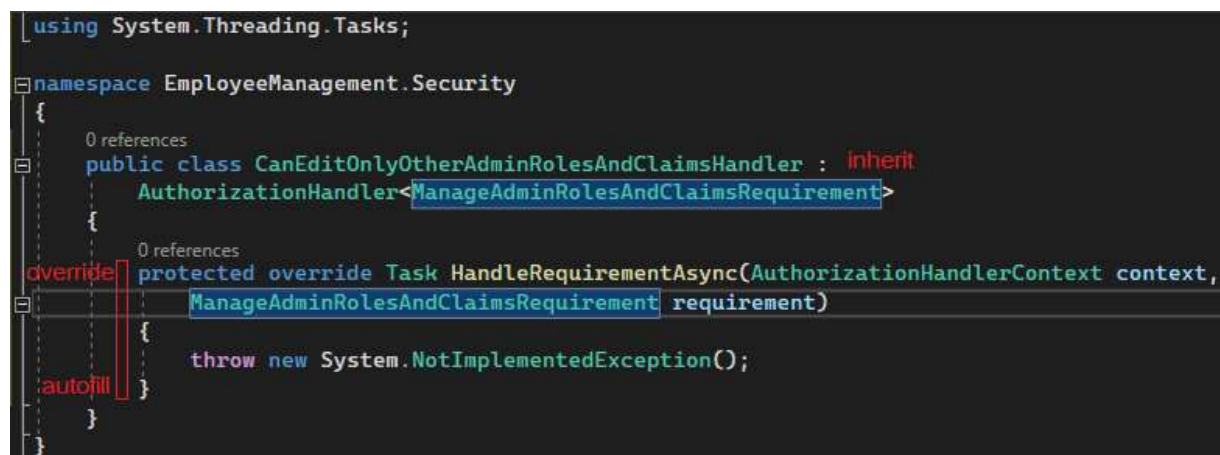
IAuthorizationRequirement не е вграден, че да предоставя наготово някакви системни функционалности, затова трябва да го създадем и персонализираме.

В папка Security -> Add -> Class -> CanEditOnlyOtherAdminRolesAndClaimsHandler.cs

Този клас ще унаследява AuthorizationHandler от тип T, където T е изискване от нашия тип ManageAdminRolesAndClaimsRequirement, т.е. този клас ще бъде аргумент като наш generic parameter.

Creating a custom authorization handler

It is in the authorization handler that we write our logic to allow or deny access to a resource like a controller action for example. To implement a handler you inherit from [AuthorizationHandler<T>](#), and implement the [HandleRequirementAsync\(\)](#) method. The generic parameter <T> on the [AuthorizationHandler<T>](#) is the type of requirement.



Записваме override + автоматично попълване.

Метод HandleRequirementAsync получава 2 входни параметъра (context, requirement = който обработваме). Обект context има .Resource property, който връща тип object, но ние го прехвърляме като AuthorizationFilterContext.

```

context = context.Resource as AuthorizationFilterContext;
context == null)
return type is object
object? AuthorizationHandlerContext.Resource { get; }
The optional resource to evaluate the AuthorizationHandlerContext.

```

Използваме .Resource за да предпазим ManageUserRoles() action + ManageUserClaims() action в AdministrationController, така че този .Resource property връща тези action methods които защитаваме като AuthorizationFilterContext. Съхраняваме този резултат в променливата var authFilterContext и ако тя е null, тогава не можем да процедираме с проверката за ауторизация, така че връщаме .CompletedTask и забранен достъп.

`AuthorizationFilterContext` is using `Microsoft.AspNetCore.Mvc.Filters`;

The following line returns the controller action being protected as the `AuthorizationFilterContext` and provides access to HttpContext, RouteData, and everything else provided by MVC and Razor Pages.

```

var authFilterContext = context.Resource as AuthorizationFilterContext;

```

If `AuthorizationFilterContext` is `NULL`, we cannot check if the requirement is met or not, so we return `Task.CompletedTask` and the **access is not authorised**.

```

if (authFilterContext == null)
{
    return Task.CompletedTask;
}

protected override Task HandleRequirementAsync(AuthorizationHandlerContext context,
    ManageAdminRolesAndClaimsRequirement requirement)
{
    .prop returns controller action being protected and provides access
    var authFilterContext = context.Resource as AuthorizationFilterContext;
    if (authFilterContext == null)
    {
        return Task.CompletedTask;    Access Denied
    }
}

```

The incoming context object has list of user claims, from this list of user claims, we search for the NameOfIdentifier Claim, because it contains the ID of the logged in user. Next we want the ID of the admin being edited. It is passed in the URL as a query string parameter. To retrieve it we are using this authFilterContext object. Through this object we have access to HTTP context, route data, query string parameters and everything else provided by MVC and Razor Pages. After we have successfully retrieved the loggedInAdminId and adminIdBeingEdited, our requirement is met and the authorization succeeds.

We check:

If the user is in the Admin role

AND

has Edit Role claim type with a claim value of true

AND

the logged-in user Id is NOT EQUAL TO the Id of the Admin user being edited

```

if (context.User.IsInRole("Admin") &&
    context.User.HasClaim(claim => claim.Type == "Edit Role" && claim.Value == "true") &&

```

```

adminIdBeingEdited.ToLower() != loggedInAdminId.ToLower())
{
    context.Succeed(requirement);
}

```

`Succeed()` method specifies that the requirement is successfully evaluated.

```

public class CanEditOnlyOtherAdminRolesAndClaimsHandler : inherit
    AuthorizationHandler<ManageAdminRolesAndClaimsRequirement>
{
    0 references
    protected override Task HandleRequirementAsync(AuthorizationHandlerContext context,
        ManageAdminRolesAndClaimsRequirement requirement)
    {
        .prop returns controller action being protected and provides access
        var authFilterContext = context.Resource as AuthorizationFilterContext;
        if (authFilterContext == null)
        {
            true
            return Task.CompletedTask;    Access Denied
        }

        else string loggedInAdminId =
            context.User.Claims.FirstOrDefault(c => c.Type == ClaimTypes.NameIdentifier).Value;
            incoming context object has list of user claims, in which we search for .NameIdentifier to provide ID
            string adminIdBeingEdited = authFilterContext.HttpContext.Request.Query["userId"];

            if (context.User.IsInRole("Admin") &&
                context.User.HasClaim(claim => claim.Type == "Edit Role" && claim.Value == "true") &&
                true
                adminIdBeingEdited.ToLower() != loggedInAdminId.ToLower())
            {
                context.Succeed(requirement);
            }

            else false
            return Task.CompletedTask;    Access Denied
        }
    }
}

```

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc.Filters;
using System.Linq;
using System.Security.Claims;
using System.Threading.Tasks;

namespace EmployeeManagement.Security
{
    public class CanEditOnlyOtherAdminRolesAndClaimsHandler :
        AuthorizationHandler<ManageAdminRolesAndClaimsRequirement>
    {
        protected override Task
        HandleRequirementAsync(AuthorizationHandlerContext context,
            ManageAdminRolesAndClaimsRequirement requirement)
        {
            var authFilterContext = context.Resource as
            AuthorizationFilterContext;
            if (authFilterContext == null)
            {
                return Task.CompletedTask;
            }

            string loggedInAdminId =
                context.User.Claims.FirstOrDefault(c => c.Type ==
            ClaimTypes.NameIdentifier).Value;

            string adminIdBeingEdited =
            authFilterContext.HttpContext.Request.Query["userId"];

```



```

        if (context.User.IsInRole("Admin") &&
            context.User.HasClaim(claim => claim.Type == "Edit Role" &&
claim.Value == "true") &&
            adminIdBeingEdited.ToLower() != loggedInAdminId.ToLower())
        {
            context.Succeed(requirement);
        }

        return Task.CompletedTask;
    }
}

```

Authorization handler registration

We register custom authorization handler in `ConfigureServices()` method of the `Startup` class

```

options.AddPolicy("DeleteRolePolicy",
    policy => policy.RequireClaim("Delete Role"));

options.AddPolicy("EditRolePolicy",
    policy => policy.RequireAssertion(context =>
        context.User.IsInRole("Admin") &&
        context.User.HasClaim(claim => claim.Type == "Edit Role" &&
            claim.Value == "true") ||
        context.User.IsInRole("Super Admin")
    ));

options.AddPolicy("AdminRolePolicy",
    policy => policy.RequireRole("Admin"));
});

services.AddScoped<IEmployeeRepository, SQLEmployeeRepository>();

```

```

options.AddPolicy("DeleteRolePolicy",
    policy => policy.RequireClaim("Delete Role"));

options.AddPolicy("EditRolePolicy",
    policy => policy.AddRequirements(new ManageAdminRolesAndClaimsRequirement());

options.AddPolicy("AdminRolePolicy",
    policy => policy.RequireRole("Admin"));
});

services.AddScoped<IEmployeeRepository, SQLEmployeeRepository>();
services.AddSingleton<IAuthorizationHandler,
    CanEditOnlyOtherAdminRolesAndClaimsHandler>();

```

```

public void ConfigureServices(IServiceCollection services)
{
    services.AddAuthorization(options =>
    {
        options.AddPolicy("EditRolePolicy", policy =>

```

```

        policy.AddRequirements(new ManageAdminRolesAndClaimsRequirement());
    });

    services.AddSingleton<IAuthorizationHandler,
        CanEditOnlyOtherAdminRolesAndClaimsHandler>();
}

```

Finally use the custom policy to protect the resources like controller action methods

```

EmployeeManagement.Controllers.AdministrationCon EditRole
[HttpGet]
[Authorize(Policy = "EditRolePolicy")]
0 references
public async Task<IActionResult> EditRole(string id)

```

Вече `[Authorize(Policy = "EditRolePolicy")]` не ни е необходим в Get + Post на `EditRole`, така че го изтриваме от тях.

Необходим ни е в Get + Post на `ManageUserRoles` action.

```

[HttpGet]
[Authorize(Policy = "EditRolePolicy")]
0 references
public async Task<IActionResult> ManageUserRoles(string userId)

```

При тестване, Админ може да редактира чужди права, но не и свои собствени.

User Roles

User

Admin

Manage Roles

Към момента сме защитили само `ManageUserRoles` action, но не и `ManageUserClaims` action, така че при `Manage Claims` бутоната правилата все още не важат, така че прилагаме същият custom policy и при него.

Get + Post

```

[Authorize(Policy = "EditRolePolicy")]
ManageUserClaims

```

Към момента, за обработката на нашия Custom Requirement имаме само един Custom Handler, но също е възможно 1 requirement да има multiple handlers.

Step - 1 : Create the Custom Requirement

```

public class ManageAdminRolesAndClaimsRequirement : IAuthorizationRequirement
{
}

```

Step - 2 : Create the Custom Handler

```
public class CanEditOnlyOtherAdminRolesAndClaimsHandler :
    AuthorizationHandler<ManageAdminRolesAndClaimsRequirement>
{
    protected override Task HandleRequirementAsync(AuthorizationHandlerContext context,
        ManageAdminRolesAndClaimsRequirement requirement)
    {
        var authFilterContext = context.Resource as AuthorizationFilterContext;
        if (authFilterContext == null)
            return Task.CompletedTask;

        string loggedInAdminId =
            context.User.Claims.FirstOrDefault(c => c.Type == ClaimTypes.NameIdentifier).Value;

        string adminIdBeingEdited = authFilterContext.HttpContext.Request.Query["userId"];

        if (context.User.IsInRole("Admin") &&
            context.User.HasClaim(claim => claim.Type == "Edit Role" && claim.Value == "true") &&
            adminIdBeingEdited.ToLower() != loggedInAdminId.ToLower())
        {
            context.Succeed(requirement);
        }

        return Task.CompletedTask;
    }
}
```

Step - 3 : Register the Custom Handler

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddAuthorization(options =>
    {
        options.AddPolicy("EditRolePolicy", policy =>
            policy.AddRequirements(new ManageAdminRolesAndClaimsRequirement()));
    });

    services.AddSingleton<IAuthorizationHandler>,
        CanEditOnlyOtherAdminRolesAndClaimsHandler>();
}
```

Step - 4 : Use the Custom Policy

```
[HttpGet]
[Authorize(Policy = "EditRolePolicy")]
public async Task<IActionResult> ManageUserRoles(string userId)
{
    // Code
    return View();
}
```